

Consumo de API

1. O que é uma API

API é a sigla para *Application Programming Interface* (Interface de Programação de Aplicações). Na prática, é um "contrato" que um sistema oferece para que outros programas possam pedir e receber dados dele, sem precisar saber como esse sistema funciona por dentro.

No dia a dia de quem trabalha com tecnologia, "consumir uma API" é uma das tarefas mais comuns: praticamente todo aplicativo de celular, site ou sistema troca informações com outros serviços dessa forma.

2. Como uma API funciona

Uma API funciona como uma ponte entre a nossa aplicação e os dados de um servidor:

1. Nossa aplicação faz uma **requisição** (pede algo).
2. O servidor processa o pedido e devolve uma **resposta**, geralmente no formato **JSON** (o mesmo formato que já usamos na Aula 10).



Nossa aplicação --- requisição ---> Servidor (dados da API)

Nossa aplicação <--- resposta ----- Servidor (dados da API)

3. A biblioteca `requests`

Para fazer requisições HTTP em Python, usamos a biblioteca externa `requests` (ela não vem instalada por padrão, então é preciso instalar com `pip install requests`).

As partes mais usadas dessa biblioteca são:

- `requests.get(url)`: envia uma requisição do tipo GET (buscar dados) para o endereço informado e devolve um objeto de resposta.
- `resposta.status_code`: código numérico que indica se a requisição deu certo. Os mais comuns:
 - **200 (OK)**: deu tudo certo, os dados estão disponíveis.
 - **404 (NOT FOUND)**: o endereço/recurso pedido não existe.
 - **500 (INTERNAL SERVER ERROR)**: ocorreu um erro do lado do servidor.
- `resposta.json()`: converte o conteúdo JSON da resposta em um dicionário (ou lista) Python, do mesmo jeito que `json.loads()`, visto na Aula 10.

Como uma requisição pode falhar por vários motivos (sem internet, CEP inexistente, servidor fora do ar), sempre envolvemos essas chamadas em `try/except` — retomando o conteúdo da Aula 08. O erro mais comum de conexão é o

```
requests.exceptions.ConnectionError
```

4. Exemplo resolvido: consultando um CEP (API ViaCEP)

A ViaCEP é uma API pública e gratuita para consultar Códigos de Endereçamento Postal (CEP) do Brasil. Basta acessar a URL `https://viacep.com.br/ws/{cep}/json/`, substituindo `{cep}` pelo CEP desejado.

```
python
"""
ATENÇÃO: necessário instalar o pacote requests
"""

import requests

try:
    cep = input('Digite o cep: ')
    response = requests.get(f'http://viacep.com.br/ws/{cep}/json/')

    if response.status_code == 200:
        dicionario = response.json()

        if 'erro' in dicionario:
            print('CEP inexistente')
        else:
            print(f"Rua: {dicionario['logradouro']}")
            print(f"Bairro: {dicionario['bairro']}")
        else:
            print(f'Erro de requisição: {response.status_code}')
except requests.exceptions.ConnectionError as erro:
    print('Erro. Erro de Conexão com API')
except Exception as erro:
    print(f'Erro: {erro}')
```

Repare nos pontos importantes:

- A API sempre devolve `status_code == 200`, mesmo quando o CEP não existe — nesse caso, o dicionário retornado contém a chave `"erro"`. Por isso, verificamos `'erro' in dicionario` antes de acessar os outros campos.

- Usamos dois blocos `except`, cada um com seu próprio apelido (`as erro`), tratando primeiro o erro específico de conexão e depois qualquer outro erro inesperado — mesma boa prática vista na Aula 08.

5. Exemplos de mercado

APIs estão por trás de praticamente qualquer produto digital que usamos. Alguns exemplos reais:

- **Preenchimento automático de endereço em checkouts de e-commerce:** lojas virtuais brasileiras usam APIs como a ViaCEP (ou serviços dos Correios) para preencher rua e bairro assim que o cliente digita o CEP — exatamente o que fizemos no exemplo acima.
- **Cotações financeiras:** aplicativos de bancos, corretoras e conversores de moeda consomem APIs de cotação (como a AwesomeAPI, que usaremos no Exercício 5) para exibir o valor do dólar, euro ou bitcoin em tempo real.
- **Previsão do tempo:** apps de clima, aplicativos de logística e até sistemas de agricultura consomem APIs meteorológicas (como a Open-Meteo, usada no Exercício 6) para tomar decisões baseadas na previsão do tempo.
- **Catálogos e integrações de e-commerce:** marketplaces e ERPs trocam catálogos de produtos entre sistemas via API — o mesmo princípio que aplicaremos hoje no nosso projeto de e-commerce.
- **Login social, mapas, pagamentos:** "Entrar com Google/Facebook", mapas dentro de apps de entrega (Google Maps API) e cobranças via Pix/cartão (APIs de gateways como Stripe, PagSeguro e Mercado Pago) são todos exemplos de consumo de API no dia a dia.

6. Exercícios principais

Os exercícios abaixo estão organizados em ordem crescente de dificuldade. Os quatro primeiros vêm da lista de exercícios da aula; os três últimos foram acrescentados a partir de pesquisa de outras APIs públicas gratuitas, para aprofundar o tema.

Exercício 1 — Municípios de uma UF

Solicite ao usuário a sigla de uma UF e utilize a API abaixo para consultar e exibir os nomes de todos os municípios da UF informada.



<http://servicodados.ibge.gov.br/api/v1/localidades/estados/{uf}/municipios>

Exercício 2 — Lista de usuários em ordem alfabética

Utilize a API abaixo para gerar uma listagem com os nomes de N usuários. Exiba a lista em ordem alfabética.



<http://randomuser.me/api/?results={n}>

Exercício 3 — Busca de receitas por ingrediente

A API abaixo retorna um JSON com informações de 50 receitas culinárias. Faça um programa que consulte a API e exiba todas as receitas que utilizam um determinado ingrediente informado pelo usuário.



<http://dummyjson.com/recipes?limit=50>

Exercício 4 — Valor total do estoque

A API abaixo retorna um JSON com as informações de 100 produtos. Faça um programa que consulte a API e informe o valor total do estoque (somatório de todos os produtos). Considere que cada produto possui um preço (`price`), um desconto (`discountPercentage`) que deve ser aplicado ao preço de cada produto e uma quantidade de itens em estoque (`stock`).



<http://dummyjson.com/products?limit=100>

Exercício 5 — Cotação de moedas (pesquisa própria)

Solicite ao usuário um par de moedas (ex.: `USD-BRL`, `EUR-BRL`) e utilize a API pública de cotações **AwesomeAPI** para exibir o nome da moeda e os valores de compra (`bid`) e venda (`ask`).



<https://economia.awesomeapi.com.br/json/last/{moedas}>

Exercício 6 — Previsão do tempo (pesquisa própria)

Solicite ao usuário a latitude e a longitude de um local e utilize a API pública **Open-Meteo** para exibir a temperatura e a velocidade do vento atuais naquele ponto.



https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}¤t_weather=true

Exercício 7 — Imagem do dia da NASA (pesquisa própria)

A NASA publica todos os dias a **APOD** (*Astronomy Picture of the Day*, "imagem astronômica

do dia"). Faça um programa que consulte a API abaixo e exiba o **título**, a **data** e o **link da imagem** do dia.



<https://science.nasa.gov/wp-json/wp/v2/apod-basic>

Dicas:

- Esta API **não exige chave**: basta fazer o `requests.get()` com a URL.
- A resposta é uma **lista** de imagens, da mais nova para a mais antiga. A primeira (posição 0) é a imagem de hoje.
- Cada imagem é um dicionário. Os campos que você vai usar são `title` (título), `date` (data) e `hdurl` (link da imagem).
- Para ver a imagem, copie o link impresso e abra no navegador.
- Existe um endereço antigo (`api.nasa.gov/planetary/apod`) que sai do ar em 01/12/2026. Use o endereço novo, indicado acima.

Desafio opcional: exiba também o crédito (`credit`) e a explicação (`explanation`) da imagem. Atenção: esses dois textos vêm com tags HTML misturadas (como `<strong>` e `<a href=...>`). Pesquise como removê-las.

7. Tema complementar: paginação de resultados de uma API

Algumas APIs têm respostas grandes demais para devolver de uma vez só, e por isso **dividem o resultado em páginas**. Nesses casos, é comum que o JSON retornado traga uma chave (ex.: `"next"`) apontando para a URL da próxima página. Quando não existe mais próxima página, essa chave vem com valor `None` / `null`.

Para percorrer todas as páginas, usamos um laço `while True`, que só é interrompido (`break`) quando a URL da próxima página deixa de existir.

Exercício complementar 1 — Listando todos os Pokémon (PokeAPI)

A PokeAPI é uma API pública que fornece acesso a dados sobre o universo Pokémon. O endpoint abaixo retorna uma lista de Pokémon, paginada através da chave `"next"`.



<http://pokeapi.co/api/v2/pokemon/>

Implemente uma aplicação para consultar essa API e listar os nomes de **todos** os Pokémon disponíveis, navegando por todas as páginas.

Exercício complementar 2 — Listando todos os personagens (Rick and Morty API, pesquisa própria)

A Rick and Morty API também pagina seus resultados, mas guarda a URL da próxima página dentro de uma chave aninhada, `dados["info"]["next"]` — uma boa forma de reforçar o mesmo conceito com um formato de resposta ligeiramente diferente.



`https://rickandmortyapi.com/api/character`

Liste os nomes de todos os personagens disponíveis, navegando por todas as páginas.

8. Evolução do projeto de fundo — E-commerce simples

Assim como nas Aulas 08, 09 e 10, a Aula 11 apenas **adiciona** funcionalidades novas ao projeto, sem alterar nada que já existia: as funções abaixo usam o conceito de consumo de API para (1) importar produtos automaticamente a partir de uma API pública de catálogo e (2) consultar um endereço de entrega a partir de um CEP.

Enunciado

1. Crie a função `importar_produtos_de_api`, que consulta a API pública `https://dummyjson.com/products` e cadastra os produtos retornados no catálogo, **reaproveitando** a função `cadastrar_produto` já existente (mesmo princípio já usado por `importar_produtos_de_arquivo`, na Aula 09). Trate erros de conexão e de requisição sem interromper o programa.
2. Crie a função `consultar_endereco_por_cep`, que consulta a API ViaCEP e devolve um dicionário com o endereço correspondente a um CEP (ou `None`, se o CEP não existir ou ocorrer algum erro).

9. APIs com chave (API key)

Até aqui, todas as APIs que usamos aceitavam qualquer requisição de qualquer pessoa. Em produtos reais, porém, é muito comum que a API queira saber **quem** está pedindo os dados. É para isso que existe a **chave de acesso** (*API key*).

9.1 O que é uma API com chave

Uma **API key** é uma string longa e única, gerada quando você se cadastra no serviço. Ela funciona como a "identidade" da sua aplicação: toda requisição precisa carregar essa chave para ser aceita.

Os provedores exigem chave por alguns motivos:

- **Controle de uso:** cada chave tem um limite de requisições (por exemplo, 1.000 por hora). Passou do limite, o servidor recusa o pedido.
- **Cobrança:** em serviços pagos, o consumo é contabilizado por chave.
- **Segurança e rastreo:** se alguém abusar do serviço, o provedor bloqueia apenas aquela chave.

A chave pode ser enviada de duas formas (cada API define a sua):

Forma	Como fica	Observação
Parâmetro na URL	<code>https://api.exemplo.com/dados?api_key=SUA_CHAVE</code>	Mais simples, mas a chave pode ficar registrada em históricos e logs
Cabeçalho (header)	<code>requests.get(url, headers={"X-API-Key": "SUA_CHAVE"})</code>	Mais seguro e comum em serviços profissionais

Com chaves, aparecem novos códigos de status no `status_code`:

- **401 (Unauthorized):** a chave não foi enviada ou não é válida.
- **403 (Forbidden):** a chave foi recusada ou não tem permissão para aquele recurso (algumas APIs usam 403 para chave inválida).
- **429 (Too Many Requests):** o limite de uso da chave foi excedido.

Regra de ouro: a chave é como uma senha. **Nunca** escreva a chave diretamente no código que será compartilhado ou enviado ao GitHub: qualquer pessoa poderia usá-la no seu lugar (e, em serviços pagos, você pode ser cobrado por isso). O caminho padrão é guardar a chave **fora do código**, em uma *variável de ambiente*, e lê-la com `os.environ` (ver seção 9.4).

9.2 Demonstração com a DEMO_KEY da NASA

A NASA mantém um portal de APIs abertas (`https://api.nasa.gov`). Vamos usar a **NeoWs** (*Near Earth Object Web Service*), mantida pelo JPL, que informa sobre **asteroides com aproximação da Terra** em um intervalo de datas. O endpoint que usaremos é:



```
https://api.nasa.gov/neo/rest/v1/feed
```

Parâmetros: `start_date` e `end_date` (formato `AAAA-MM-DD`, com intervalo máximo de 7 dias) e `api_key` (a chave).

O portal oferece uma chave pública de demonstração, chamada `DEMO_KEY`, para quem quer explorar as APIs antes de se cadastrar. Como ela é compartilhada por muita gente, tem limites de uso bem menores que os de uma chave pessoal. Por ser pública, não há problema em escrevê-la no código, e por isso é ideal para a primeira demonstração.

Por que não usamos a "foto do dia" (APOD)? Ela é a API mais famosa da NASA, mas, segundo a própria documentação, em 10/09/2026 ela foi migrada para um novo endereço

(`science.nasa.gov/wp-json/wp/v2/apod-basic`), com formato de resposta diferente, e a versão antiga sai do ar em 01/12/2026. Isso é uma lição valiosa: **APIs mudam**. Antes de usar uma API, consulte a documentação oficial e teste o código; e sempre trate os erros. Você consome a APOD, no endereço novo, no Exercício 7.

Novidade no código: em vez de montar a URL com `?api_key=...` na mão, passamos o argumento `params` para o `requests.get()`. A biblioteca monta a *query string* por nós.

```
python
import requests

try:
    data = input('Data (AAAA-MM-DD): ')
    resposta = requests.get(
        'https://api.nasa.gov/neo/rest/v1/feed',
        params={'start_date': data, 'end_date': data, 'api_key': 'DEMO_KEY'},
    )

    if resposta.status_code == 200:
        dados = resposta.json()
        asteroides = dados['near_earth_objects'].get(data, [])
        print(f'Asteroides com aproximação em {data}: {len(asteroides)}')
        for asteroide in asteroides:
            print(f"- {asteroide['name']}")
    elif resposta.status_code == 429:
        print('Limite de requisições excedido. Tente novamente mais tarde.')
    else:
        print(f'Erro de requisição: {resposta.status_code}')
except requests.exceptions.ConnectionError as erro_conexao:
    print('Erro de conexão com a API')
except Exception as erro:
    print(f'Erro: {erro}')
```

Estrutura do JSON retornado:

- `element_count`: quantidade total de asteroides no intervalo.
- `near_earth_objects`: **dicionário** em que cada chave é uma data (`"2026-10-05"`, por exemplo) e cada valor é uma **lista** de asteroides daquela data. Por isso usamos `.get(data, [])`: se não houver asteroides na data, devolvemos uma lista vazia em vez de gerar erro.
- Cada asteroide é um dicionário com campos como `name` (nome), `is_potentially_hazardous_asteroid` (`True`/`False`), `estimated_diameter` (diâmetro estimado) e `close_approach_data` (lista com os dados da aproximação: velocidade, distância etc.).

Se a turma inteira usar a `DEMO_KEY` ao mesmo tempo, é provável que o limite estoure e a API responda **429**. Esse é um bom momento para mostrar o tratamento desse erro e para motivar a criação de uma chave pessoal.

9.3 Guia: como criar uma chave real na NASA

O cadastro é gratuito e leva poucos minutos.

1. Acesse **<https://api.nasa.gov>** no navegador (o formulário de cadastro exige JavaScript habilitado).
2. Vá até a seção **Generate API Key**.
3. Preencha as informações solicitadas (nome e e-mail) e clique em **Signup**.
4. A chave aparece na tela logo após o cadastro e também é enviada para o seu e-mail. **Guarde esse e-mail**, pois ele é o seu backup.
5. Copie a chave. Ela será usada na próxima etapa (variável de ambiente).

Informações importantes:

- Com a chave pessoal, o limite padrão é de **1.000 requisições por hora**. Se o limite for excedido, a chave fica temporariamente bloqueada.
- Se perder a chave, o portal tem a opção **Recover API Key**.
- É possível criar mais de uma chave, se necessário.

9.4 Guia: guardando a chave em uma variável de ambiente

Uma **variável de ambiente** é um valor guardado no sistema operacional, fora do seu código. O Python consegue lê-lo, e a chave nunca aparece no arquivo `.py`.

Defina a variável `NASA_API_KEY` no terminal, **antes** de executar o programa:

Sistema	Comando
Windows (PowerShell)	<code>\$env:NASA_API_KEY="sua_chave_aqui"</code>
Windows (Prompt de Comando)	<code>set NASA_API_KEY=sua_chave_aqui</code>
Linux / macOS	<code>export NASA_API_KEY="sua_chave_aqui"</code>

Atenção: esses comandos valem **apenas para a janela de terminal em que foram digitados**. Execute o programa na mesma janela. Se abrir um novo terminal, defina a variável novamente. (Para gravar a variável de forma permanente no Windows existe o comando `setx`, mas ele só vale para terminais abertos depois dele.)

Para ler a variável no Python usamos o módulo `os`, que já conhecemos da Aula 10:

- `os.environ.get("NOME")`: devolve o valor da variável ou `None` se ela não existir (sem gerar erro).

Agora o programa lê a chave do ambiente e não usa mais a `DEMO_KEY`. Repare em três boas práticas:

- Se a variável não estiver definida, o programa **avisa e para**, em vez de usar uma chave padrão em silêncio.
- Tratamos os erros típicos de API com chave: 401/403 (chave inválida) e 429 (limite excedido).
- A lógica da consulta fica dentro de uma função, que devolve `None` em caso de erro, e o programa principal decide o que exibir.

Dois recursos novos do `requests`:

- `timeout=10`: se o servidor não responder em 10 segundos, a requisição é abandonada (evita o programa ficar travado esperando).
- `resposta.headers`: dicionário com os **cabeçalhos da resposta**. O portal da NASA costuma informar nele quantas requisições ainda restam para a chave, no campo `X-RateLimit-Remaining`. Como esse campo pode não vir, usamos `.get()` e só exibimos se existir.



python

```
import os
import requests

def obter_chave_nasa() -> str | None:
    """Lê a chave da API da NASA a partir de uma variável de ambiente.

    Returns:
        A chave configurada em NASA_API_KEY ou None se a variável não existir.
    """
    return os.environ.get("NASA_API_KEY")

def consultar_asteroides(chave: str, data: str) -> list[dict[str, object]] | None:
    """Consulta os asteroides com aproximação da Terra em uma data (NeoWs).

    Args:
        chave: chave de acesso da API.
        data: data da consulta, no formato AAAA-MM-DD.

    Returns:
        Uma lista de dicionários, um por asteroide (vazia se não houver
        nenhum na data), ou None em caso de erro.
    """
    parametros = {"start_date": data, "end_date": data, "api_key": chave}

    try:
        resposta = requests.get(
            "https://api.nasa.gov/neo/rest/v1/feed",
            params=parametros,
            timeout=10,
        )

        restantes = resposta.headers.get("X-RateLimit-Remaining")
        if restantes is not None:
            print(f"Requisições restantes para esta chave: {restantes}")

        if resposta.status_code == 200:
            dados = resposta.json()
            return dados["near_earth_objects"].get(data, [])
        if resposta.status_code in (401, 403):
            print("Chave inválida ou sem permissão.")
        elif resposta.status_code == 429:
```

```

        print("Limite de requisições excedido. Tente mais tarde.")
    else:
        print(f"Erro de requisição: {resposta.status_code}")
    return None
except requests.exceptions.ConnectionError as erro_conexao:
    print(f"Erro de conexão com a API: {erro_conexao}")
    return None
except Exception as erro:
    print(f"Erro: {erro}")
    return None

if __name__ == "__main__":
    chave = obter_chave_nasa()

    if chave is None:
        print("Defina a variável de ambiente NASA_API_KEY antes de executar.")
    else:
        data_digitada = input("Data (AAAA-MM-DD): ").strip()
        asteroides = consultar_asteroides(chave, data_digitada)

        if asteroides is not None:
            if len(asteroides) == 0:
                print("Nenhum asteroide encontrado nessa data.")
            for asteroide in asteroides:
                diametro = asteroide["estimated_diameter"]["meters"]["estimated_diameter_max"]
                aproximacao = asteroide["close_approach_data"][0]
                velocidade = float(aproximacao["relative_velocity"]["kilometers_per_hour"])
                perigoso = "Sim" if asteroide["is_potentially_hazardous_asteroid"] else "Não"

                print("-" * 40)
                print(f"Nome: {asteroide['name']}")
                print(f"Díâmetro máximo estimado: {diametro:.0f} m")
                print(f"Velocidade: {velocidade:.0f} km/h")
                print(f"Potencialmente perigoso: {perigoso}")

```

Teste também os erros: defina `NASA_API_KEY` com um valor inventado e veja a mensagem de chave inválida; apague a variável e veja o aviso de variável não definida; digite uma data em formato errado e veja o erro de requisição.

9.6 Sugestão de outra API com chave: Alpha Vantage

A **Alpha Vantage** fornece dados do mercado financeiro (ações, câmbio e criptomoedas). Combina bem com os exemplos de mercado desta aula e com o projeto de e-commerce

(preços, moedas).

Por que é uma boa segunda API com chave:

- A chave é gerada na hora, com um formulário curto (nome, organização e e-mail), em `https://www.alphavantage.co/support/#api-key`.
- O plano gratuito permite cerca de **25 requisições por dia** (e 5 por minuto). É pouco, então cada aluno deve usar a **sua** chave e evitar `requests.get()` dentro de laços.
- Ela mostra uma diferença importante em relação à NASA: a chave é enviada **apenas como parâmetro** `apikey` **na URL** (não aceita cabeçalho) e os erros (como limite diário excedido ou chave inválida) costumam voltar com `status_code == 200`, trazendo no JSON uma mensagem em chaves como `"Information"` ou `"Error Message"`. Ou seja: só olhar o `status_code` não basta, é preciso olhar também o conteúdo.

Exemplo: cotação atual de uma ação (variável de ambiente `ALPHAVANTAGE_API_KEY`):

```
python
```

```

import os
import requests

chave = os.environ.get("ALPHAVANTAGE_API_KEY")

if chave is None:
    print("Defina a variável de ambiente ALPHAVANTAGE_API_KEY.")
else:
    try:
        simbolo = input("Símbolo da ação (ex.: IBM): ").upper()
        resposta = requests.get(
            "https://www.alphavantage.co/query",
            params={"function": "GLOBAL_QUOTE", "symbol": simbolo, "apikey": chave},
            timeout=10,
        )

        if resposta.status_code == 200:
            dados = resposta.json()
            if "Information" in dados or "Error Message" in dados:
                print(dados.get("Information") or dados.get("Error Message"))
            elif not dados.get("Global Quote"):
                print("Símbolo não encontrado.")
            else:
                cotacao = dados["Global Quote"]
                print(f"Símbolo: {cotacao['01. symbol']}")
                print(f"Preço: {cotacao['05. price']}")
                print(f"Variação: {cotacao['10. change percent']}")
            else:
                print(f"Erro de requisição: {resposta.status_code}")
        except requests.exceptions.ConnectionError as erro_conexao:
            print("Erro de conexão com a API")
        except Exception as erro:
            print(f"Erro: {erro}")

```

Ideias de atividade com a Alpha Vantage: pedir um símbolo ao usuário e mostrar a cotação; comparar o preço de dois símbolos; e, como desafio, gravar a cotação consultada em um arquivo JSON (juntando as Aulas 10 e 11).